

## **Lecture 1**

# **Introduction to Computer Architecture, Number System and Logic Gates**

### **Text Book:**

Digital Design and Computer Architecture,  
David M.H and Sarah L.H, Morgan Kaufmann, Elsevier,  
(ISBN: 978-0-12-394424-5)

# What is computer architecture?

It is the design of computers including

- Instruction set
- Hardware components
- System organization

# Computer Architecture (1)

- Computer Architecture has two essential parts:
  - **Instruction-set architecture (ISA)** or **Machine language**
  - **Hardware-system architecture (HSA)**
- **ISA:** the specifications that determine how to implement machine-language of the system that will help the compiler/OS designer to interact with the machine- *Control Unit design*.
- **HSA:** computer hardware design such as CPU, Memory, Interface, etc.
- Two computers with the same ISA/machine language will execute the same program.

# Computer Architecture (2)

- **Computer Family:** a set of implementations that share the same ISA (Instruction set, called Machine Language).
- **Compatibility:** is the ability of different computers to run the same programs.
  - **Upward Compatibility:** high performance can run lower-performance in the same family.
  - **Downward Compatibility:** converse to upward.
  - **Forward compatibility:** software compatibility between one family and the later.

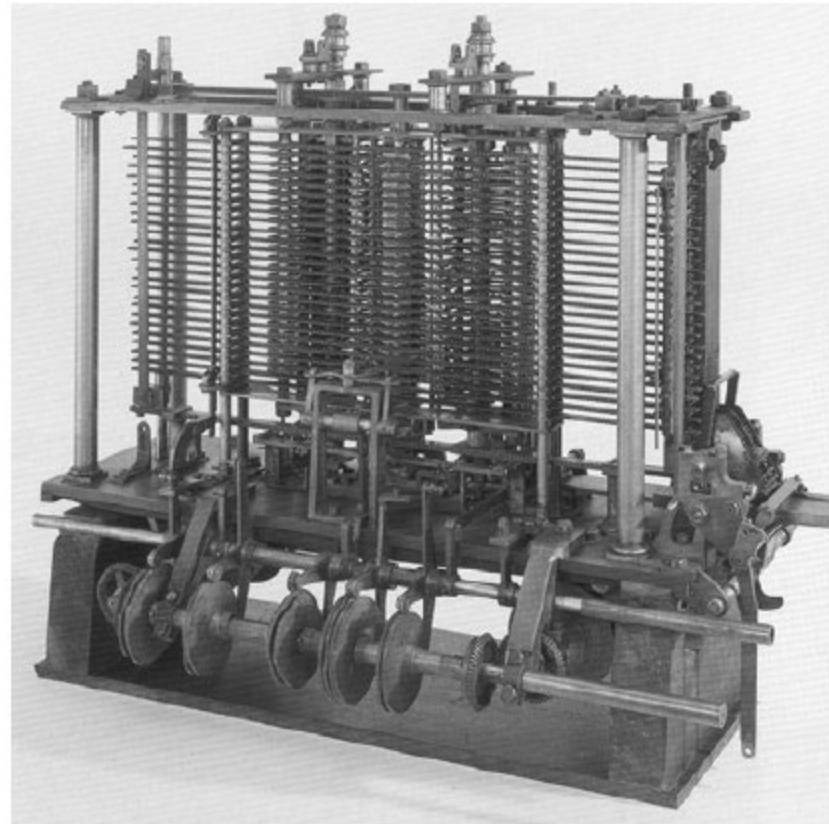
# Historical Perspectives(1)

- **Charles Babbage** (1792-1871) called father of modern computers.
  - Built difference engine (1820-1822)
  - Later it was known as '**Analytical engine**'.
- **John Atanasoff**(1903) designed an instrument for solving algebraic equations.

# Analytical engine

**Babbage's Analytical Engine, under construction at the time of his death in 1871**

(image courtesy of Science Museum/Science and Society Picture Library)



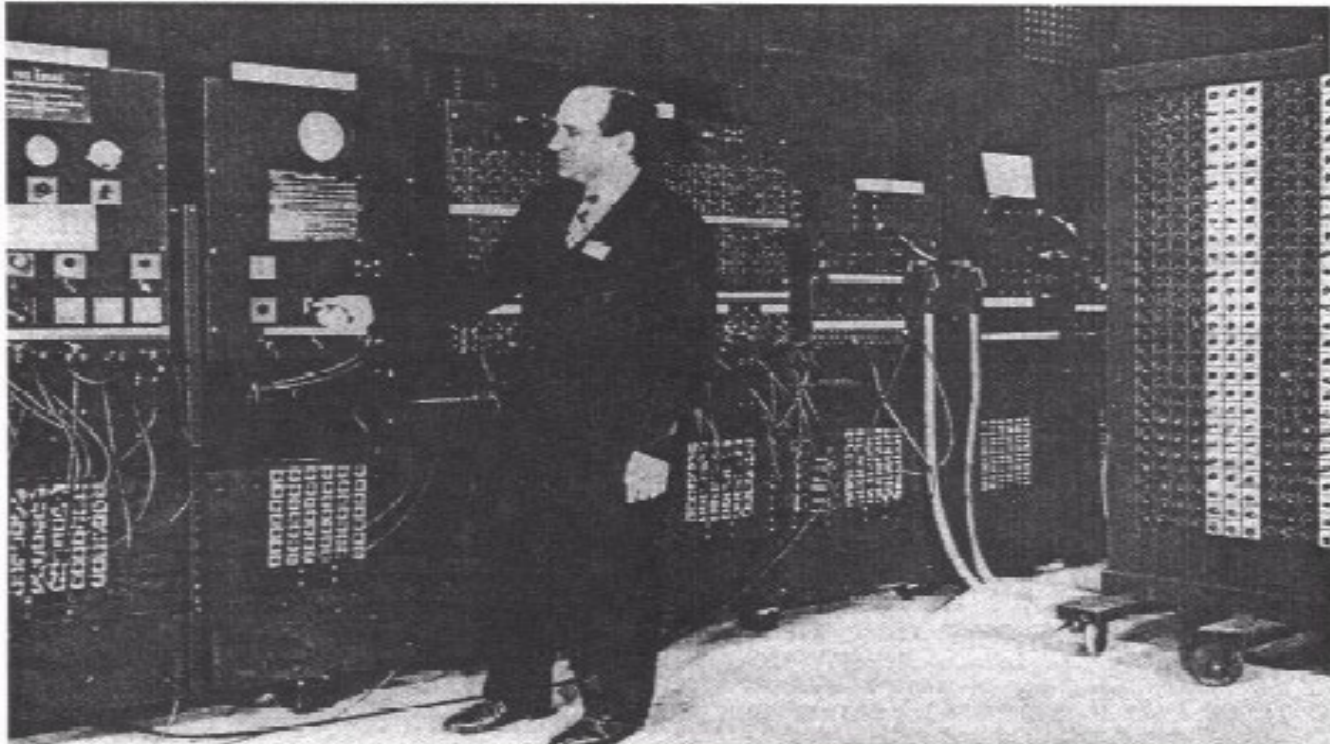
# Historical Perspectives(3)

- **Eckert** (1919-1995) and **Mauchly** (1908-1980) developed **ENIAC** (Electronic Numerical Integrator and Computer) at the Moore school of the University of Pennsylvania.
  - **Not a stored program computer**

# ENIAC

- ENIAC

Hugh electronic computer (over 18,000 vacuum tubes) developed during World War II - did not have stored program (programming done by setting switches).



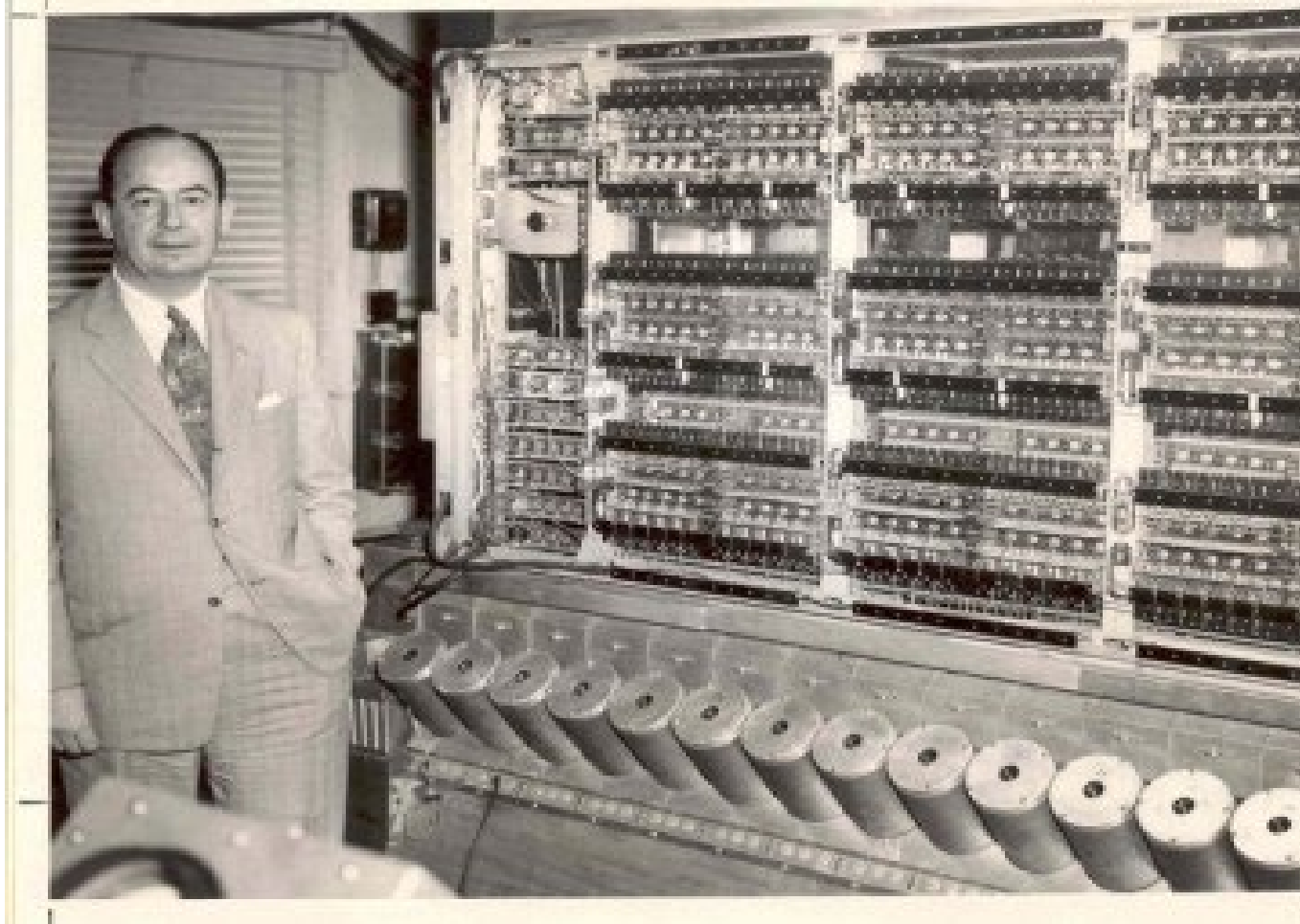
ENIAC with Eckert. Program entered using switches on lower part of equipment



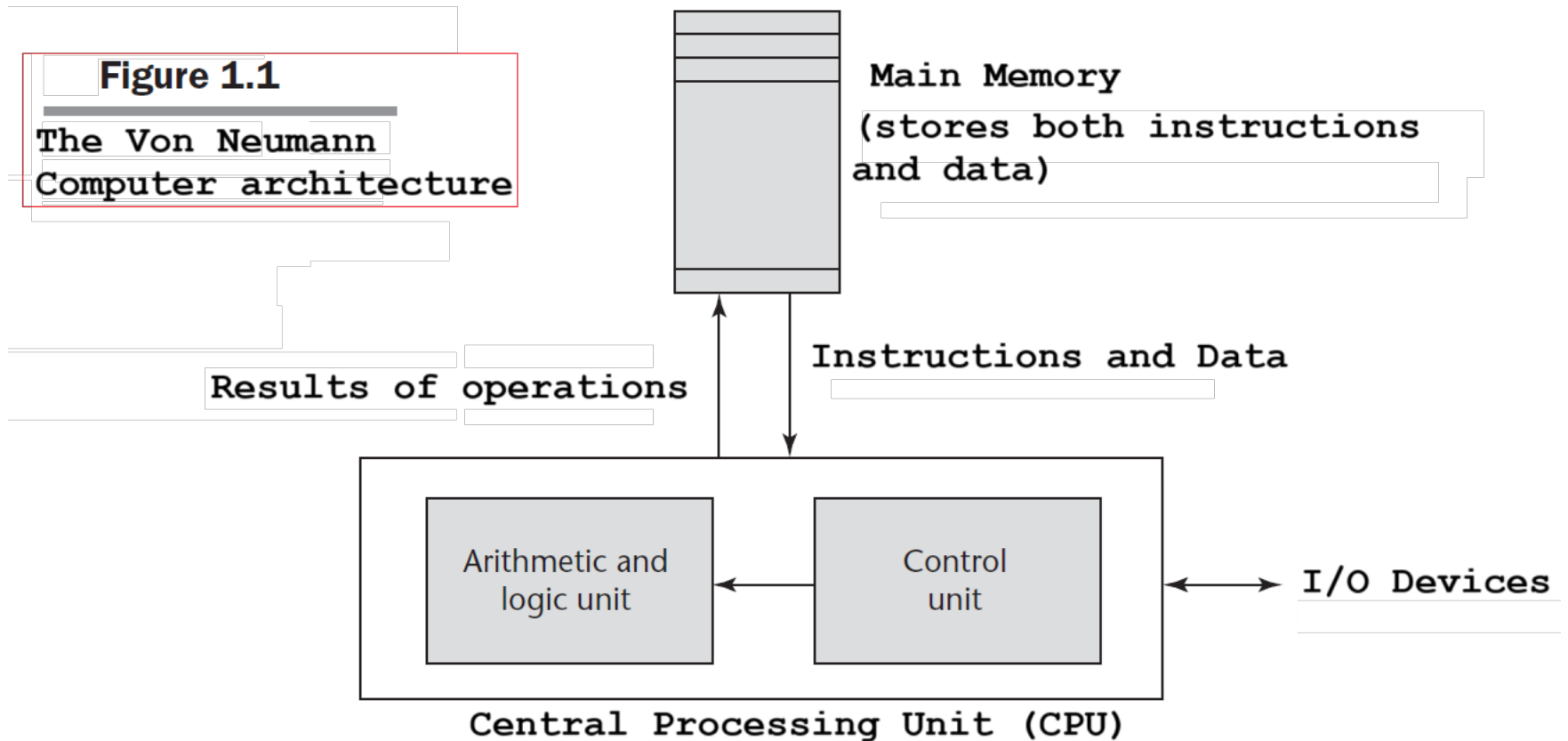
# Historical Perspectives(4)

- In 1944, **Von Neumann** was attracted to ENIAC project. That resulted to the generation of the proposal of a **stored program computer, EDVAC** (Electronic Discrete Variable Automatic Computer).
- Based on the idea from Von Neumann, Maurice built world's first stored program computer, **EDSAC** (Electronic Delay Storage Automatic Calculator) and which used mercury delay lines for its memory. The EDSAC became operational in 1949.
- In 1946, **Von Neumann** together with **Julian** designed **IAS machine** at the Princeton Institute for Advanced study. It had a total of 1024 40-bit words and roughly 10 times faster than ENIAC.

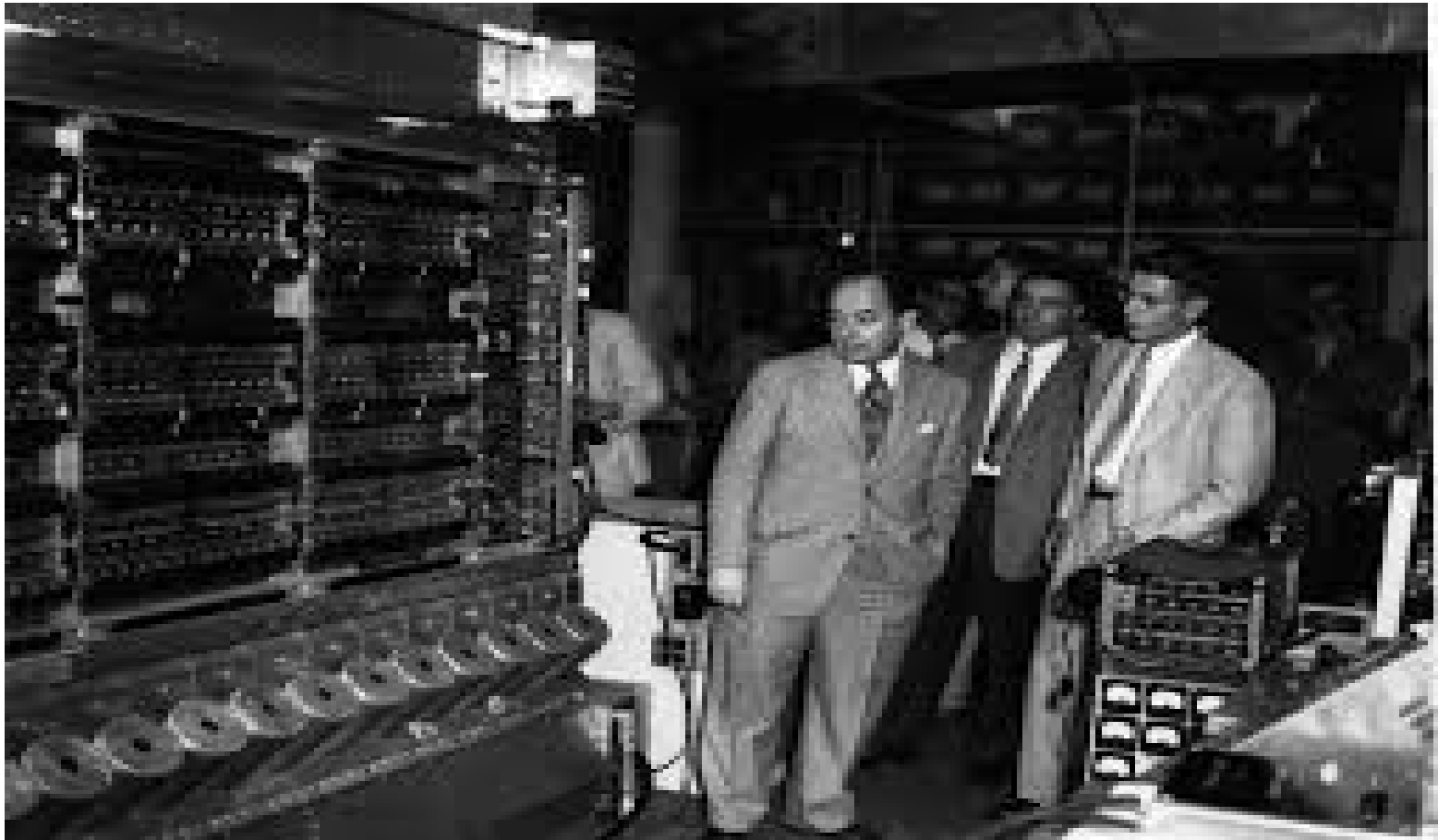
# 1946, Von Neumann Machine (IAS Machine)



# 1946, Von Neumann Machine (IAS Machine)

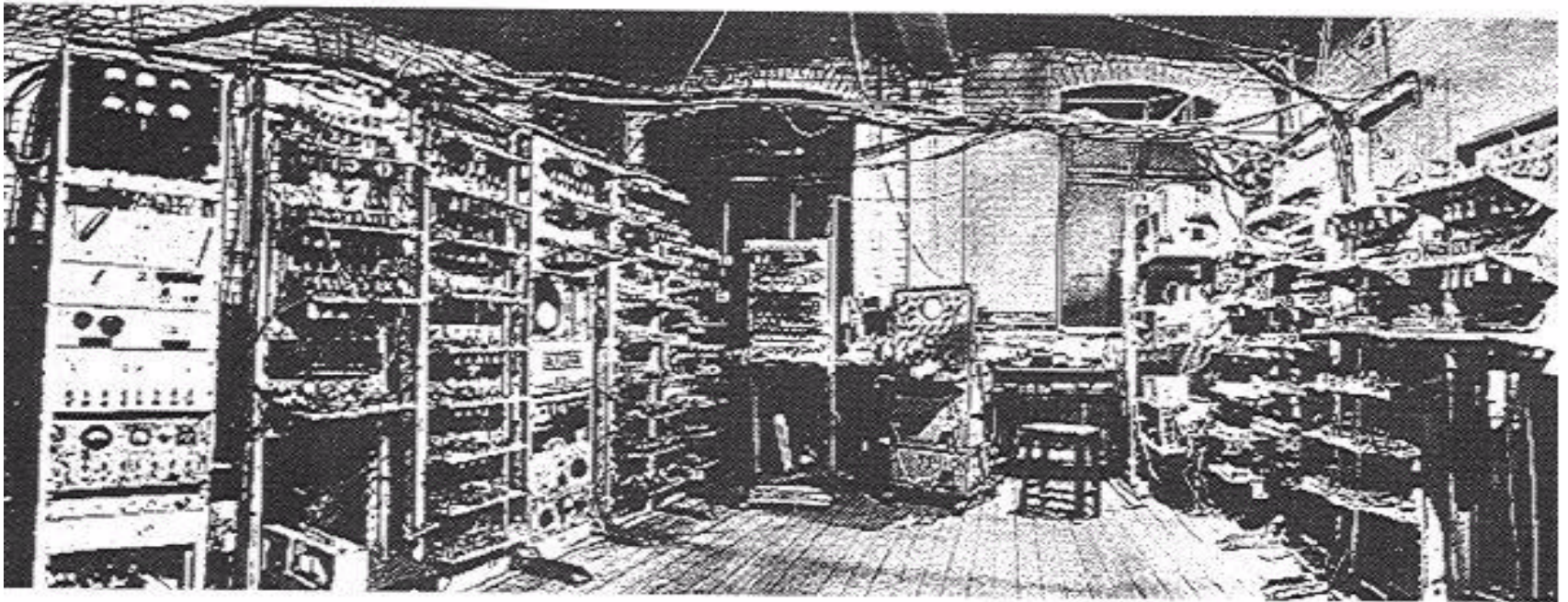


# 1946, Von Neumann Machine (IAS Machine)



# Historical Perspectives(5)

The world's first electronic *stored program* computer

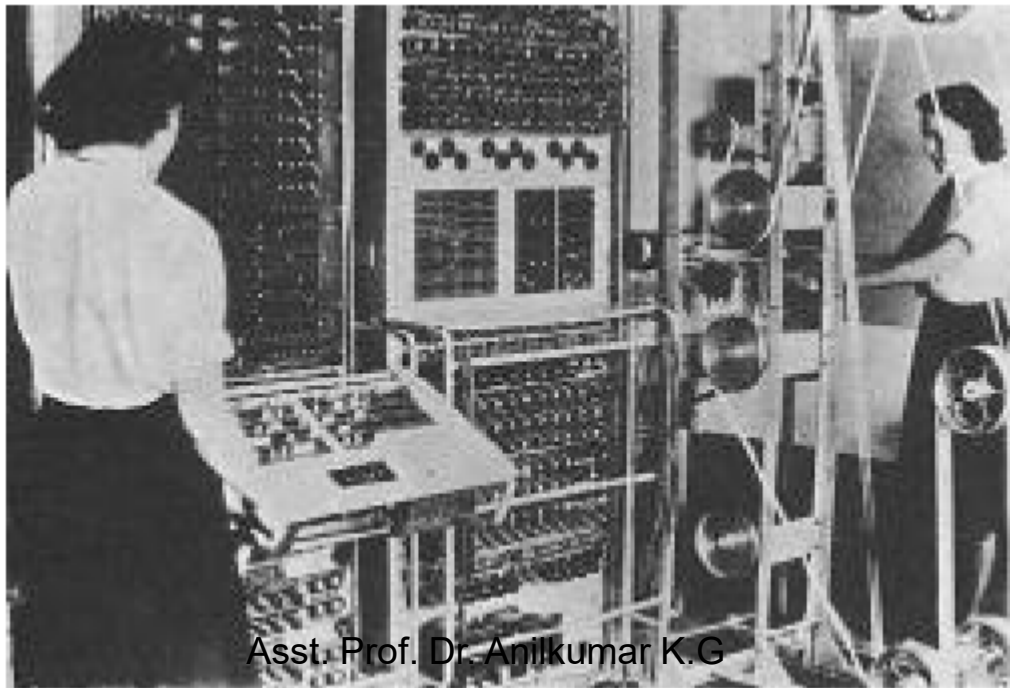


The Manchester University Mark I, operational June 21, 1948

# Historical Perspectives(2)

- **Colossus**

- British Government supported for breaking enciphered German messages during WWII
- First operational 1943 and kept secret until 1975.



Asst. Prof. Dr. Anilkumar K.G

# Historical Perspectives(6)

- Computer technology has evolved since late 1940s.
- **First generation** computers were one-of-a-kind laboratory machines.
- Most of the early machines used **vacuum-tube** technology.
- Early 1960s computer containing tens of thousands of **transistors** instead of vacuum-tube, these were the **second-generation** computers.

# Historical Perspectives(7)

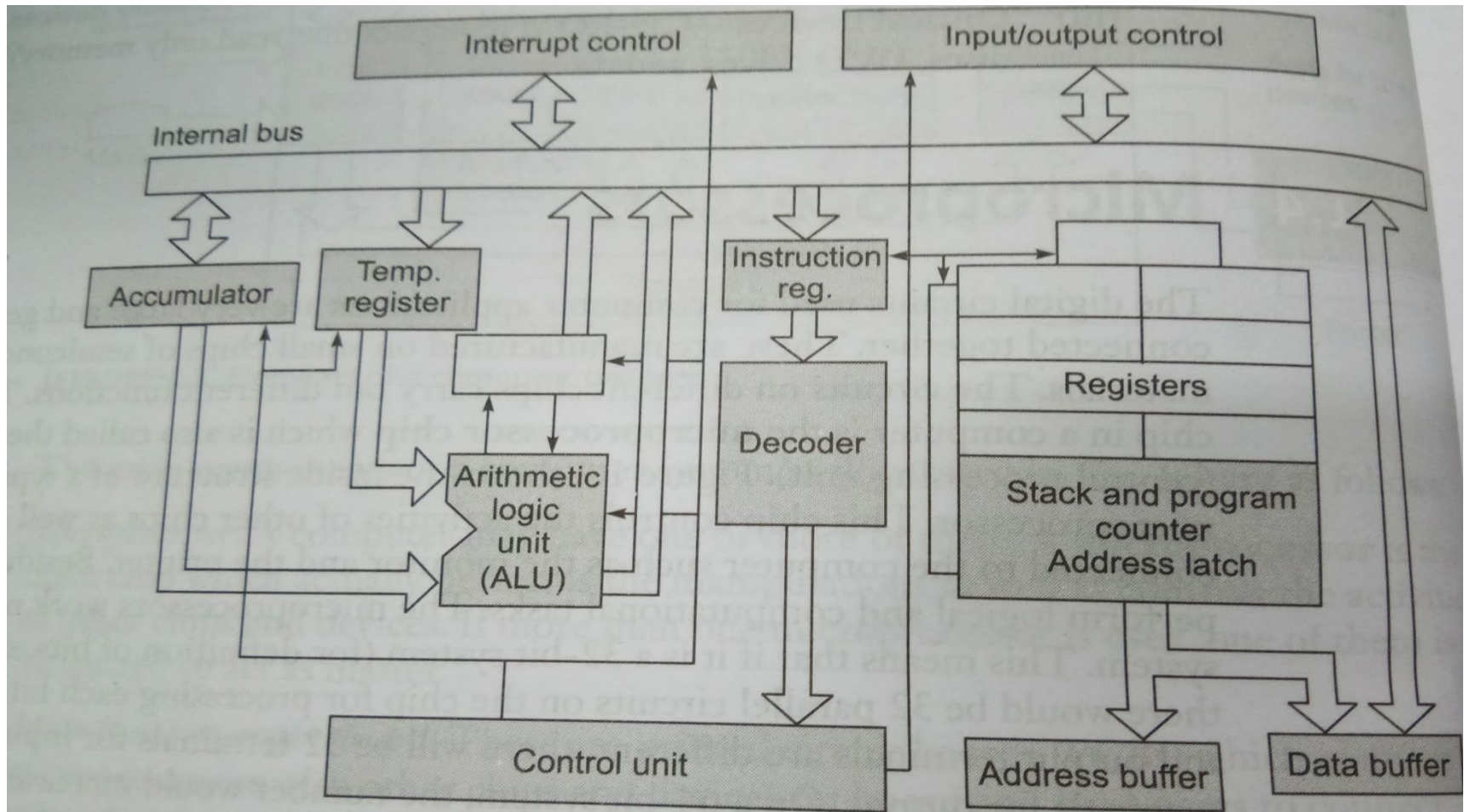
## Stretch Computer

1961 IBM - Designed to “stretch” technology/performance to its limits - germanium transistor based - much faster than earlier tube based systems (IBM 704/5.) e.g. 1 microsecond add compared to 84 microsecond add.

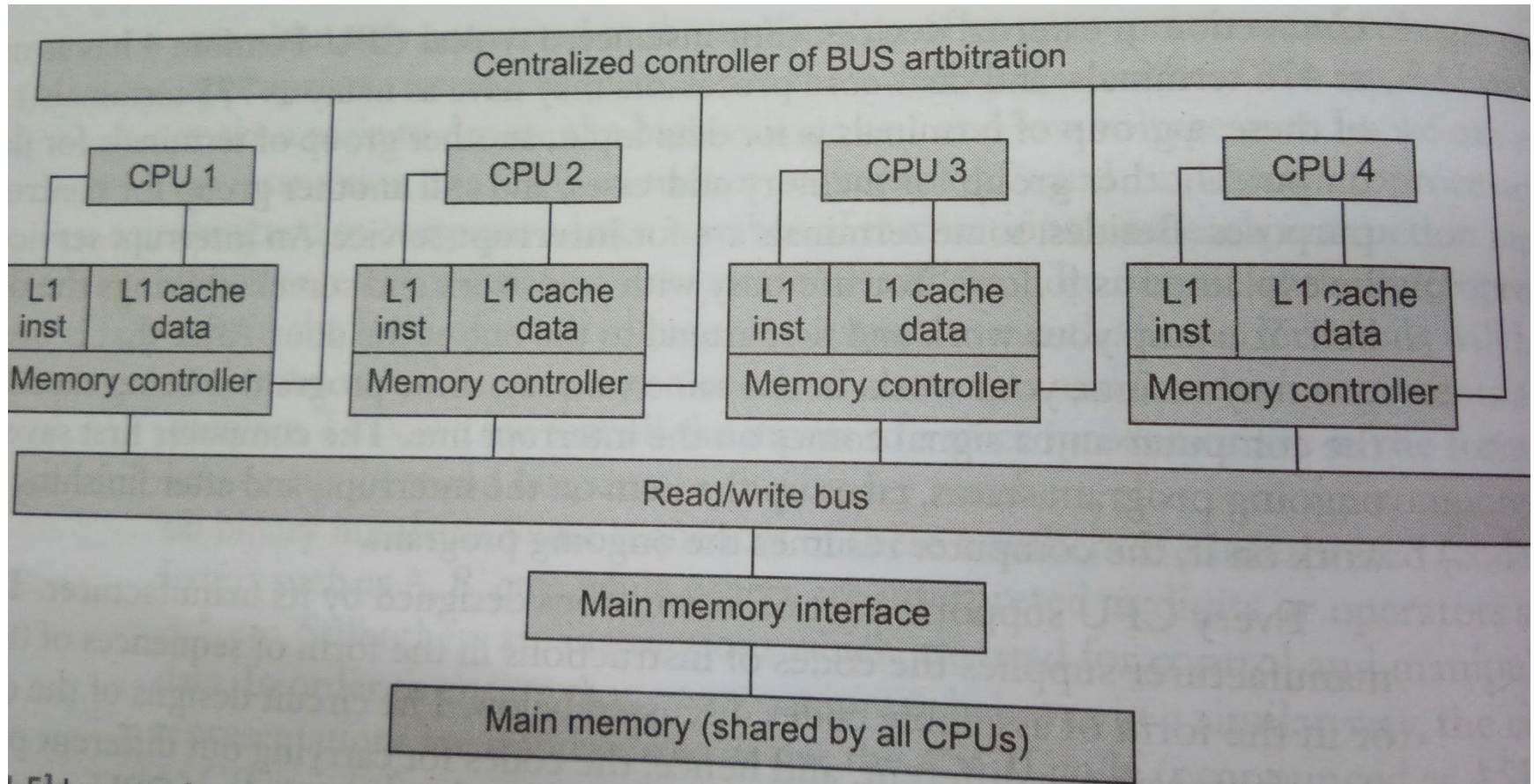




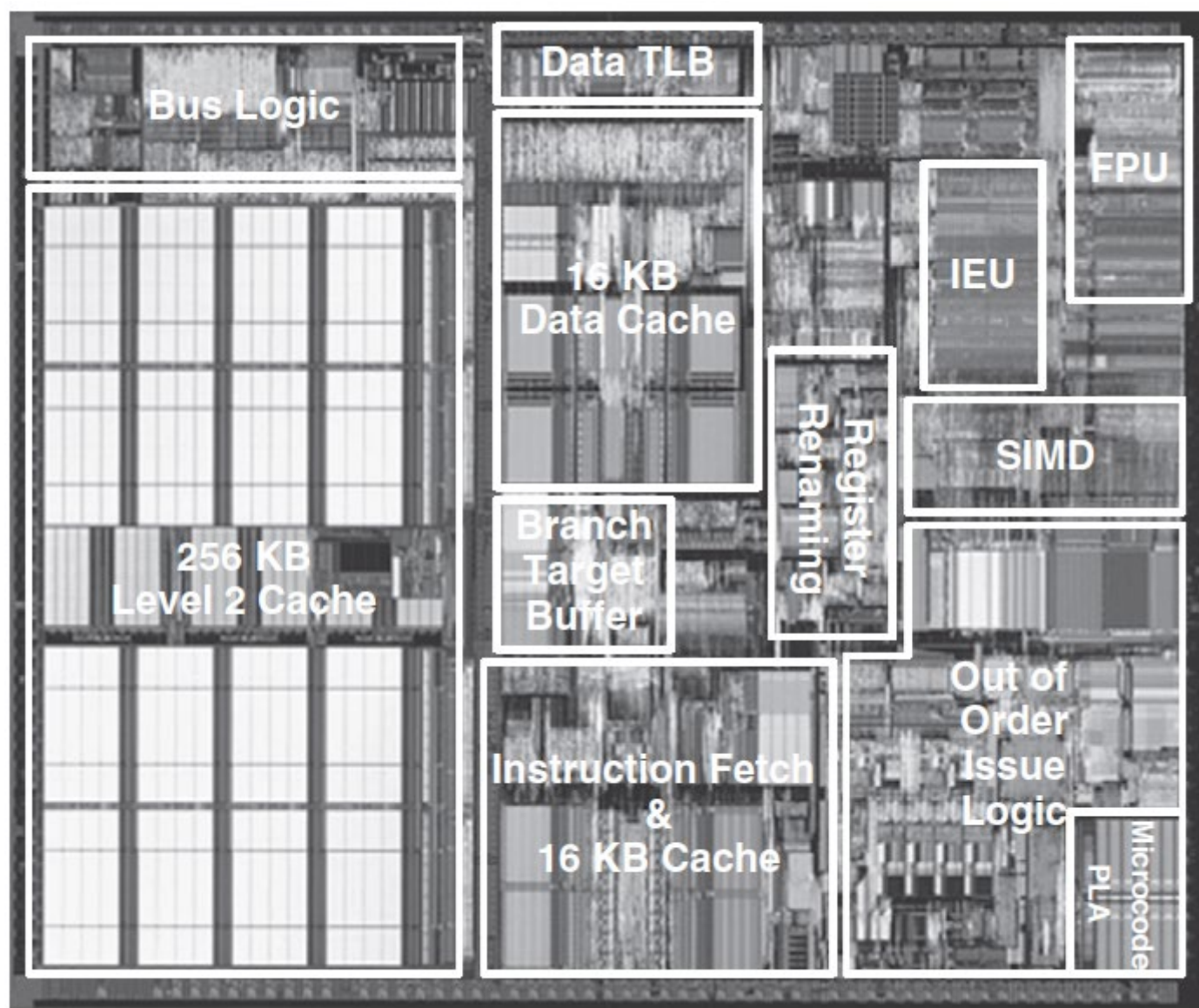
# Internal Structure of Intel 8085 CPU



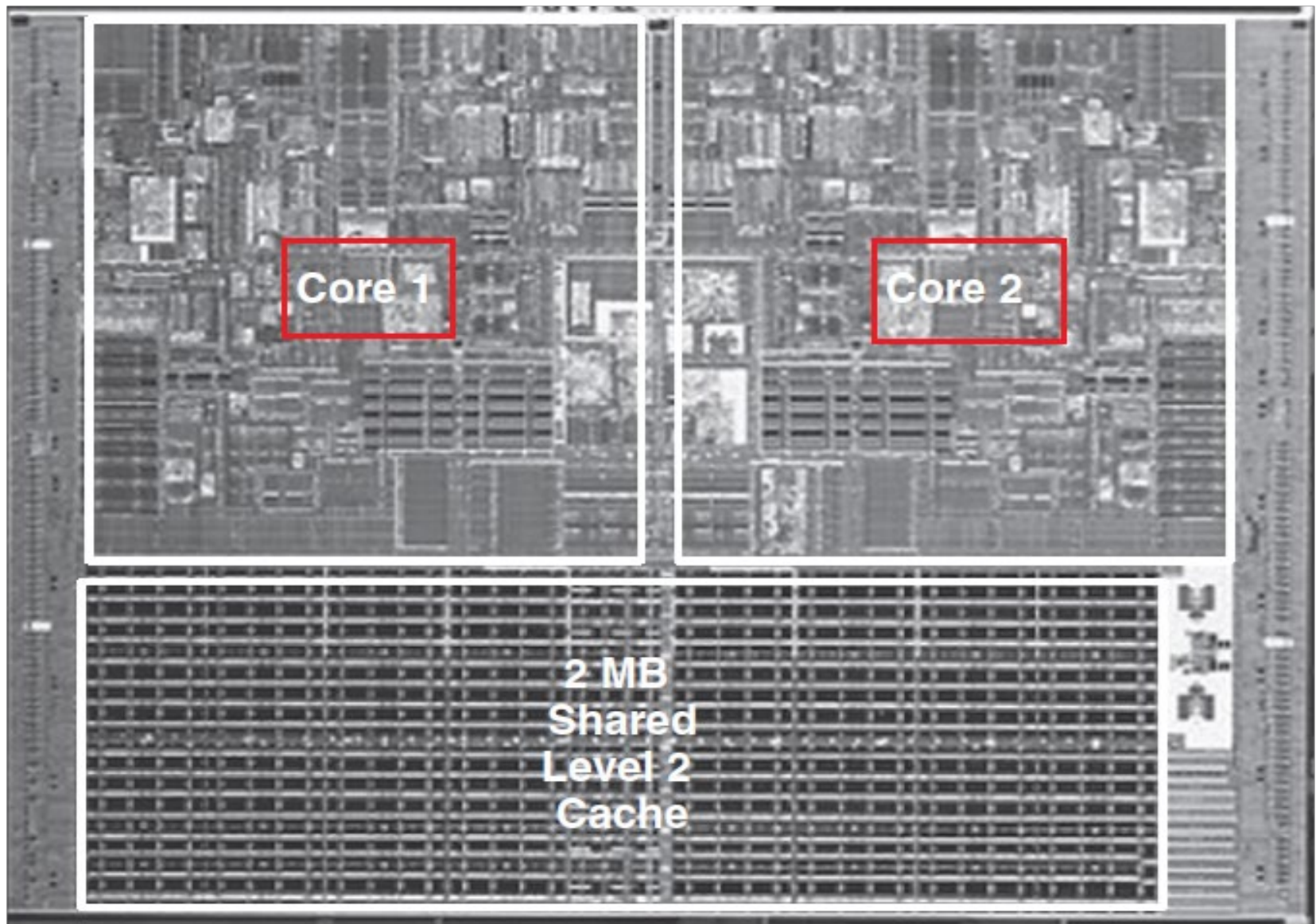
# Structure of a Quad-core Processor





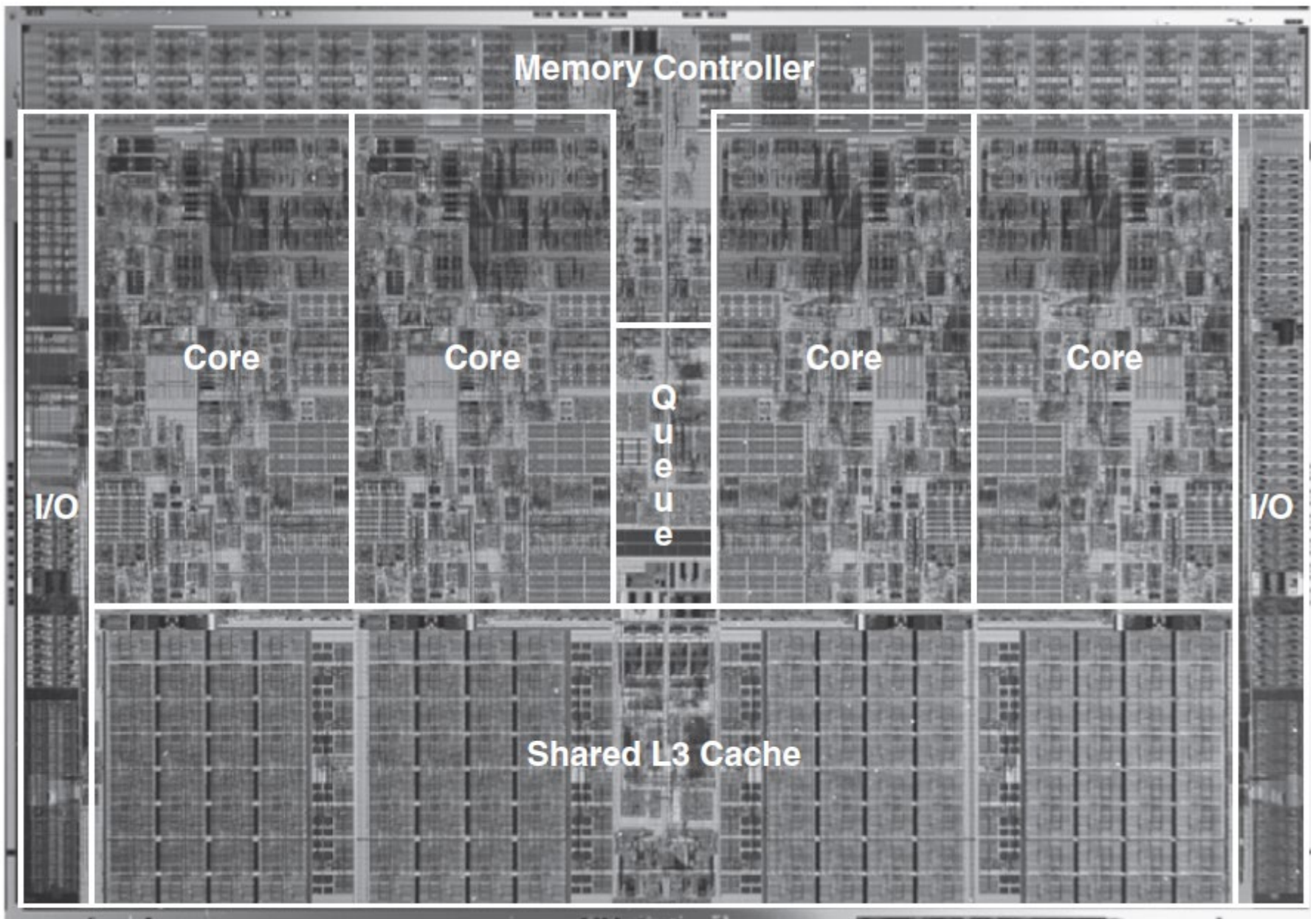


**Figure 7.77** Pentium III microprocessor chip



**Figure 7.79** Core Duo microprocessor chip





**Figure 7.80** Core i7 microprocessor chip

# Number Systems

- **Number systems** use positional notation to represent value
- Four such systems used in practice are
  - **Decimal** number system
  - **Binary** number system
  - **Hexadecimal** number system
  - **Octal** number system

# Decimal Number System

- The **decimal** number system has **ten unique symbols**:  
0, 1, 2, 3, 4, 5, 6, 7, 8, and 9
- It is also called **Base 10** number system
  - the system of counting has ten symbols
  - Its **radix** or **base** is said to be 10
- For example
  - $(3472)_{10} = 3 * 10^3 + 4 * 10^2 + 7 * 10^1 + 2 * 10^0$   
 $= 3000 + 400 + 70 + 2$
  - $(536.15)_{10} = 5 * 10^2 + 3 * 10^1 + 6 * 10^0 + 1 * 10^{-1} + 5 * 10^{-2}$

# Decimal Number System

1's column  
10's column  
100's column  
1000's column

$$9742_{10} = 9 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

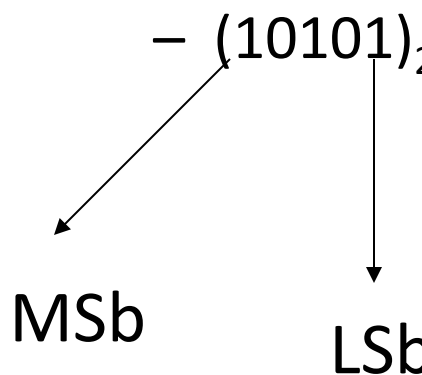
                    nine                seven                four                two  
                    thousands    hundreds        tens                ones

**Representation  
of a decimal number**



# Binary Number System

- The **binary number** system has only two symbols: 0 and 1
- Hence the **radix** or **base** is said to be 2
- For example

$$\begin{aligned} - (10101)_2 &= 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 \\ &= 16 + 0 + 4 + 0 + 1 \\ &= 21 \text{ (the **decimal equivalent** of } 101101_2\text{)} \end{aligned}$$


MSb

LSb

MSb: Most Significant bit

LSb: Least Significant bit

Asst. Prof. Dr. Anilkumar K.G

# Binary Number System

- What is the **decimal equivalent** of  $11001.011_2$  ?

$$11001.011_2$$

$$= 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} + 1 * 2^{-3}$$

$$= 16 + 8 + 0 + 0 + 1 + 0 + 0.25 + 0.125$$

$$= 25.375$$

# Decimal to Binary Conversion

- Convert the following **decimal** numbers into their **binary** equivalent:
  - 17
  - 251
  - 4372
  - 0.71 (do not need to use IEEE floating point format! Show its 4-bit binary equivalent)
  - 0.567 (do not need to use IEEE floating point format! Show its 4-bit binary equivalent)

# Binary to Decimal Conversion

- Convert the following **binary** numbers into their **decimal** equivalent:
  - $10110_2$
  - $1110111_2$
  - $10.011_2$
  - $11.1011_2$

# Binary to Decimal Conversion

Conversion of a binary  
number to decimal

1's column  
2's column  
4's column  
8's column  
16's column

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

one sixteen    no eight    one four    one two    no one

| Decimal Equivalent | Binary Equivalent |
|--------------------|-------------------|
| 0                  | 0000              |
| 1                  | 0001              |
| 2                  | 0010              |
| 3                  | 0011              |
| 4                  | 0100              |
| 5                  | 0101              |
| 6                  | 0110              |
| 7                  | 0111              |
| 8                  | 1000              |
| 9                  | 1001              |
| 10                 | 1010              |
| 11                 | 1011              |
| 12                 | 1100              |
| 13                 | 1101              |
| 14                 | 1110              |
| 15                 | 1111              |

# Binary Addition

carries → 11

$$\begin{array}{r} 1011 \\ + 0011 \\ \hline 1110 \end{array}$$



# Binary Subtraction

- In a binary system, subtraction is based on 2's complement (two's complement) operation of its values. For example,
  - If  $A$  and  $B$  are two binary numbers, then  $(A-B)$  is represented as  $(A + 2's \text{ complement of } B)$
- What is 2's complement?
  - 2's complement of a binary number is 1's complement of its bits + 1

# Hexadecimal Number System

- **Base-16** number system is called **hexadecimal number system**
  - The term '*hexadecimal*' comes from the combination of two words *hex* (means '6') and *decimal* (means '10')
  - 16 characters exists in the hexadecimal set: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F
  - This number system is most important in digital systems because it is easily to convert to and from binary

### Hexadecimal number system

| Hexadecimal Digit | Decimal Equivalent | Binary Equivalent |
|-------------------|--------------------|-------------------|
| 0                 | 0                  | 0000              |
| 1                 | 1                  | 0001              |
| 2                 | 2                  | 0010              |
| 3                 | 3                  | 0011              |
| 4                 | 4                  | 0100              |
| 5                 | 5                  | 0101              |
| 6                 | 6                  | 0110              |
| 7                 | 7                  | 0111              |
| 8                 | 8                  | 1000              |
| 9                 | 9                  | 1001              |
| A                 | 10                 | 1010              |
| B                 | 11                 | 1011              |
| C                 | 12                 | 1100              |
| D                 | 13                 | 1101              |
| E                 | 14                 | 1110              |
| F                 | 15                 | 1111              |

# Binary to Hexadecimal Conversion

- Convert  $11011110101110_2$  into hexadecimal

|      |      |      |      |
|------|------|------|------|
| 0011 | 0111 | 1010 | 1110 |
| ↓    | ↓    | ↓    | ↓    |
| 3    | 7    | A    | E    |

$= 37AE_{16} = 37AEH$

# Hexadecimal to Decimal Conversion

- Convert  $A59C_{16}$  or A59CH into decimal

$$\begin{aligned}A59CH &= A * 16^3 + 5 * 16^2 + 9 * 16^1 + C * 16^0 \\&= 40960 + 1280 + 144 + 12 \\&= 42396\end{aligned}$$

- Convert  $326_{10}$  into hexadecimal
- Convert  $3216_{10}$  into hexadecimal

# Hexadecimal to Decimal Conversion

Conversion of a  
hexadecimal number to decimal

256's column  
16's column  
1's column

$$2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two  
two hundred  
fifty six's

fourteen  
sixteens

thirteen  
ones

# Hexadecimal to Binary Conversion

- Convert F37AH (or  $F37A_{16}$ ) into Binary

$F37AH = 1111\ 0011\ 0111\ 1010$

- Convert  $23E0.B2_{16}$  into Binary

$23E0.B2H = 0010\ 0011\ 1111\ 0000 . 1011\ 0010$

### Hexadecimal number system

| Hexadecimal Digit | Decimal Equivalent | Binary Equivalent |
|-------------------|--------------------|-------------------|
| 0                 | 0                  | 0000              |
| 1                 | 1                  | 0001              |
| 2                 | 2                  | 0010              |
| 3                 | 3                  | 0011              |
| 4                 | 4                  | 0100              |
| 5                 | 5                  | 0101              |
| 6                 | 6                  | 0110              |
| 7                 | 7                  | 0111              |
| 8                 | 8                  | 1000              |
| 9                 | 9                  | 1001              |
| A                 | 10                 | 1010              |
| B                 | 11                 | 1011              |
| C                 | 12                 | 1100              |
| D                 | 13                 | 1101              |
| E                 | 14                 | 1110              |
| F                 | 15                 | 1111              |



# Octal Number System

- The octal number system has 8 symbols: 0, 1, 2, 3, 4, 5, 6, and 7
  - Hence it is called Base-8 number system

- Convert  $(231)_8$  into decimal

$$\begin{aligned}(231)_8 &= 2 * 8^2 + 3 * 8^1 + 1 * 8^0 \\ &= 128 + 24 + 1 = 153\end{aligned}$$

- Convert  $1111011110101_2$  into octal

$$\begin{array}{ccccccccc} & = & 001 & 111 & 011 & 110 & 101 & & \\ & & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & & \\ & = & 1 & 7 & 3 & 6 & 5 & = & (17365)_8 \end{array}$$

# Byte and Nibble

- A group of 8-bits called a **byte**.
  - Bits means binary digits (either 1 or 0)
- A group of 4-bits or half byte is called a **nibble**.
- **lsb** (Least Significant Bit) and **msb** (Most Significant Bit)

101100  
└─┘ └─┘  
most least  
significant significant  
bit bit

- **LSB** (Least Significant Byte) and **MSB** (Most Significant Byte)

DEAFDAD8  
└─┘ └─┘  
most least  
significant significant  
byte byte

# Data types

- **Sign and magnitude:**
  - **Sign bit**, usually the farthest to the left, designates the sign of the number
  - **Magnitude**: the remaining bits designate The value of  $n$ -bit signed-magnitude
  - The value zero has both a positive and negative representation (drawback!)
- **2's complement:**
  - Is 1's complement of a number + 1
  - Has only one representation for value zero
  - The choice for most computers
  - Achieve by adding 1's complement of a number to 1

# Two's Complement numbers

- 2's Complement of a binary number =  
1's Complement of its bits + 1

Example:

---

## **Example 1.10** TWO'S COMPLEMENT REPRESENTATION OF A NEGATIVE NUMBER

Find the representation of  $-2_{10}$  as a 4-bit two's complement number.

**Solution:** Start with  $+2_{10} = 0010_2$ . To get  $-2_{10}$ , invert the bits and add 1. Inverting  $0010_2$  produces  $1101_2$ .  $1101_2 + 1 = 1110_2$ . So  $-2_{10}$  is  $1110_2$ .

# 2's Complement

---

## **Example 1.11** VALUE OF NEGATIVE TWO'S COMPLEMENT NUMBERS

Find the decimal value of the two's complement number  $1001_2$ .

**Solution:**  $1001_2$  has a leading 1, so it must be negative. To find its magnitude, invert the bits and add 1. Inverting  $1001_2 = 0110_2$ .  $0110_2 + 1 = 0111_2 = 7_{10}$ . Hence,  $1001_2 = -7_{10}$ .

---

## **Example 1.12** ADDING TWO'S COMPLEMENT NUMBERS

Compute (a)  $-2_{10} + 1_{10}$  and (b)  $-7_{10} + 7_{10}$  using two's complement numbers.

**Solution:** (a)  $-2_{10} + 1_{10} = 1110_2 + 0001_2 = 1111_2 = -1_{10}$ . (b)  $-7_{10} + 7_{10} = 1001_2 + 0111_2 = 10000_2$ . The fifth bit is discarded, leaving the correct 4-bit result  $0000_2$ .

---

# Sign Extension

When a two's complement number is extended to more bits, the sign bit must be copied into the most significant bit positions. This process is called *sign extension*. For example, the numbers 3 and -3 are written as 4-bit two's complement numbers 0011 and 1101, respectively. They are sign-extended to seven bits by copying the sign bit into the three new upper bits to form 0000011 and 1111101, respectively.

# Data types (cont.)

- **BCD** (Binary Coded Decimal):
  - use 4 bits encode one decimal digit (0-9) by  $0000_2$  to  $1001_2$
  - The sequence of digits represent number
  - The codes  $1010_2$  to  $1111_2$  are generally not allowed  
Such case decimal adjustment is needed (add +6 to convert into BCD)
    - Example,  $345D = 001101000101$

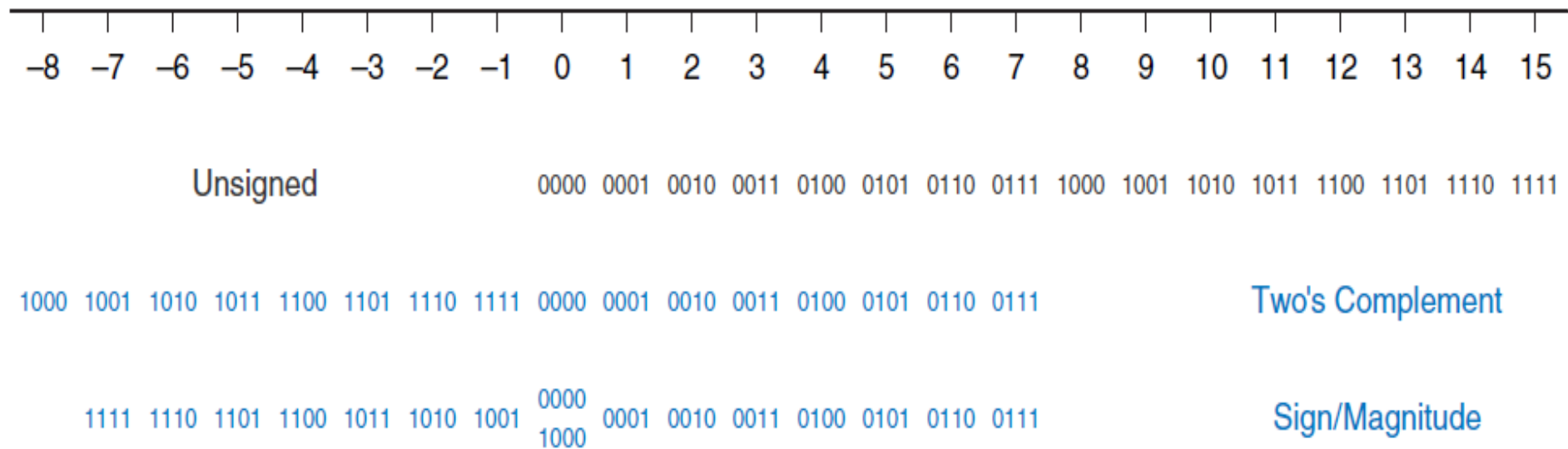
# Data types (cont.)

- **Decimal adjust:**

- Assume a 8-bit data type (let  $D_1$  is upper 4-bit part and  $D_0$  is the lower 4-bit part),
- If  $D_0 > 9$  or a carry out from bit 3, then add 00000110.
- if  $D_1 > 9$  or a carry out from bit 7, from the first add or from step 1, add 01100000.
  - Example: 56+67, 81+52



# Comparison of Number System



**Figure 1.11** Number line and 4-bit binary encodings

# Units of Data

- **Byte:** basic unit information, 8 bits, can hold single character or small integer.
- **Half word:** 16-bit (in some cases this information is different)
- **Word size:** is generally the size of its operational registers.
  - A 32-bit computer has 32-bit registers size and a word consists of 4 bytes (32 bits)
- **Double word:** two words long. (32-bit computer has 64-bit of double word)
- **Quad words:** four words long 128bit (32x4).

# Units of Data(cont.)

- **Single precision:** full word numeric representation
- **Double precision:** two word numeric representation
- One byte of storage is usually sufficient to hold one character. The most common character representation is
  - **ASCII** (American Standard Code for Information Interchange) use in personal computer
  - **EBCDIC**(Extended Binary Coded Decimal Interchange Code) use in IBM mainframes

# Floating-point Data

- Computer represent floating-point numbers in a way that resembles scientific notation-in four parts: a ***sign***, a ***mantissa*** (or ***significant***), a ***radix*** (or ***exponent base***) and an ***exponent***.
  - $976,000,000,000,000 = 9.76 \times 10^{14}$
  - $0.000,000,000,0000976 = 9.76 \times 10^{-14}$**Scientific notation**
- Some computer mantissa is integer but most computer mantissa is fraction.
- The ***radix*** are nearly always powers of 2, whereas in scientific notation the radix is 10.

# Floating-point Data (cont.)

- The bits of a word encode the *sign*, *mantissa* (or *significant*), and *exponent*, and the hardware assumes the radix.
- Floating-point number **F**, let **S** be the value of sign bit (0 if positive, 1 if negative), **M** the mantissa, **E** the Exponent and **R** the radix

Value F is  $(S) \times M \times R^E$

# Floating-point Data (cont.)

- For example, the decimal value **+18.5** can be represented as  **$+1.85 \times 10^1$**
- The value  **$+1.85 \times 10^1$**  can further be represented as  **$1.15625 \times 16$**  is **normalized into  $+1.15625 \times 2^4$**
- The binary equivalent of the mantissa part (.15625) is 00101000 hence  **$+1.15625 \times 2^4$**  is  **$+1.00101000 \times 2^4$**  .
- **$+1.00101000 \times 2^4$  cannot be converted into  $+0.000101000 \times 2^5$  or  $+0.0100101000 \times 2^6$  or .....**
- The combination of three components make the floating point number:  
**number = sign, mantissa x base<sup>exponent</sup>**

# Floating-point Data (cont.)

- The floating point numbers are not unique.
- To make the representation unique the computer need to **normalize the number**
- **Number normalization:** By repeatedly decreasing the exponent by 1 and shifting the mantissa left by 1 digit until the most significant digit is non-zero. The result is normalized number.
- ***Most computers normalize their floating-point number***

# Floating-point Data (cont.)

- Typical 32-bit Floating point format

| Sign  | Biased exponent | Fraction (significant) |
|-------|-----------------|------------------------|
| 1-bit | 8-bit           | 23-bit                 |

**32-bit representation**

Examples:

1.  $1.638125 \times 2^{20} = 0\ 10010011\ 101000110\dots\dots$

2.  $-1.638125 \times 2^{20} =$

3.  $1.638125 \times 2^{-20} =$

4.  $-1.638125 \times 2^{-20} =$

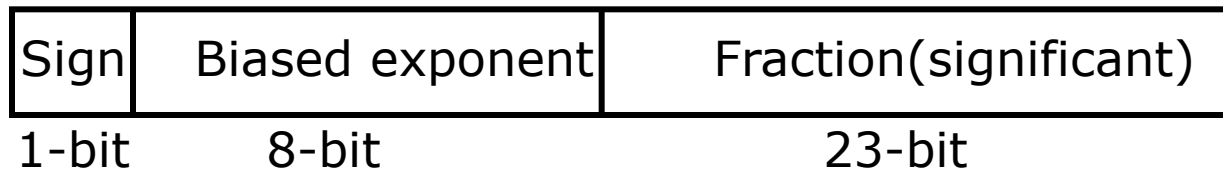


# IEEE Floating-point Standard

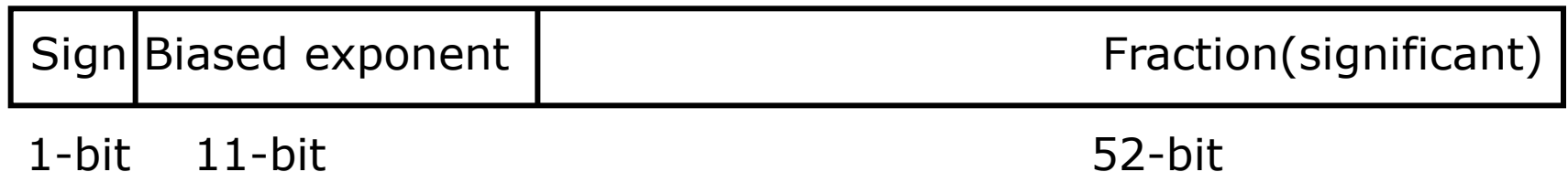
- The mantissa field assumes that binary point is to the left of the first bit
- The value of the exponent bias is 127 for 32-bit numbers and 1023 for 64-bit numbers
- Assumes a normalized mantissa with a value between  $1.000\dots0_2$  and  $1.111\dots1_2$ ; elides the leading bit

# IEEE Floating-point Standard (cont.)

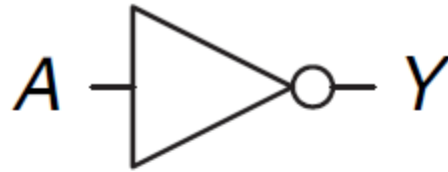
- Single format (32-bit)



- Double format(64-bit)



# Logic Gates

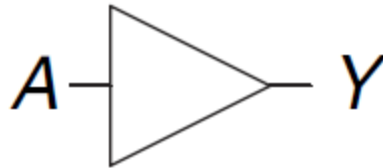


$$Y = \bar{A}$$

| $A$ | $Y$ |
|-----|-----|
| 0   | 1   |
| 1   | 0   |

**NOT gate**

# Logic Gates

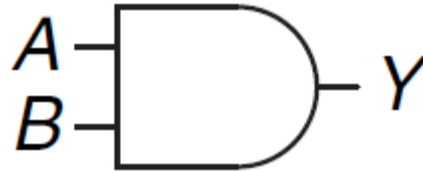


$$Y = A$$

| $A$ | $Y$ |
|-----|-----|
| 0   | 0   |
| 1   | 1   |

**Buffer**

# Logic Gates

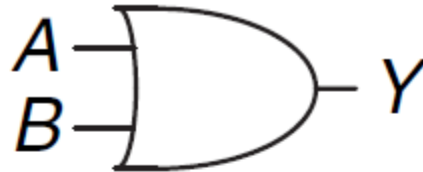


$$Y = AB$$

| <i>A</i> | <i>B</i> | <i>Y</i> |
|----------|----------|----------|
| 0        | 0        | 0        |
| 0        | 1        | 0        |
| 1        | 0        | 0        |
| 1        | 1        | 1        |

**AND gate**

# Logic Gates

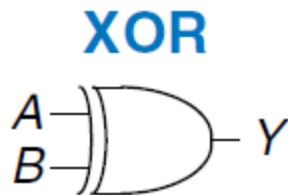


$$Y = A + B$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

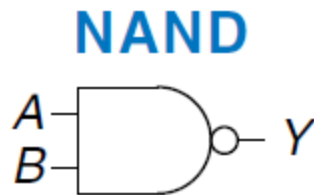
**OR gate**

# Logic Gates



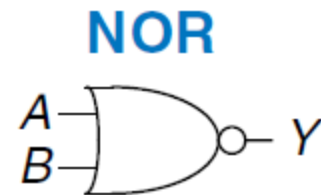
$$Y = A \oplus B$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



$$Y = \overline{AB}$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

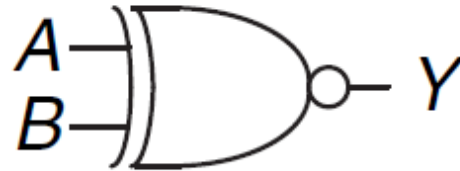


$$Y = \overline{A + B}$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**More two-input logic gates**

# Logic Gates



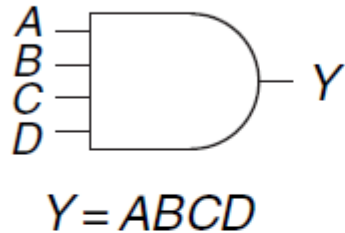
$$Y = \overline{A \oplus B}$$

| <i>A</i> | <i>B</i> | <i>Y</i> |
|----------|----------|----------|
| 0        | 0        |          |
| 0        | 1        |          |
| 1        | 0        |          |
| 1        | 1        |          |

**XNOR gate**



# Multiple Input Logic Gates



**Figure 1.21** Four-input AND gate

| <i>A</i> | <i>C</i> | <i>B</i> | <i>D</i> | <i>Y</i> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 0        | 1        | 0        |
| 0        | 0        | 1        | 0        | 0        |
| 0        | 0        | 1        | 1        | 0        |
| 0        | 1        | 0        | 0        | 0        |
| 0        | 1        | 0        | 1        | 0        |
| 0        | 1        | 1        | 0        | 0        |
| 0        | 1        | 1        | 1        | 0        |
| 1        | 0        | 0        | 0        | 0        |
| 1        | 0        | 0        | 1        | 0        |
| 1        | 0        | 1        | 0        | 0        |
| 1        | 0        | 1        | 1        | 0        |
| 1        | 1        | 0        | 0        | 0        |
| 1        | 1        | 0        | 1        | 0        |
| 1        | 1        | 1        | 0        | 0        |
| 1        | 1        | 1        | 1        | 1        |

**Figure 1.22** Four-input AND truth table

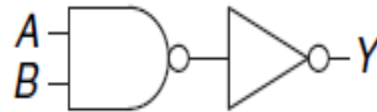
# Multiple Input Logic Gates



$$Y = \overline{A + B + C}$$

| A | B | C | Y |
|---|---|---|---|
| 0 | 0 | 0 |   |
| 0 | 0 | 1 |   |
| 0 | 1 | 0 |   |
| 0 | 1 | 1 |   |
| 1 | 0 | 0 |   |
| 1 | 0 | 1 |   |
| 1 | 1 | 0 |   |
| 1 | 1 | 1 |   |

**Figure 1.19** Three-input NOR gate



**Figure 1.37** Two-input AND gate schematic

It is impossible to build an AND gate with a single CMOS gate. However, building NAND and NOT gates is easy. Thus, the best way to build an AND gate using CMOS transistors is to use a NAND followed by a NOT, as shown in Figure 1.37.

# Logic family-Design

Four major logic families that predominated from the 1970's through the 1990's are Transistor-Transistor Logic (TTL), Complementary Metal-Oxide-Semiconductor Logic (CMOS, pronounced sea-moss), Low Voltage TTL Logic (LVTTTL), and Low Voltage CMOS Logic (LVCMOS). Their logic levels are compared in Table 1.4. Since then, logic families have balkanized with a proliferation of even lower power supply voltages. Appendix A.6 revisits popular logic families in more detail.

**Table 1.4** Logic levels of 5 V and 3.3 V logic families

| Logic Family | $V_{DD}$      | $V_{IL}$ | $V_{IH}$ | $V_{OL}$ | $V_{OH}$ |
|--------------|---------------|----------|----------|----------|----------|
| TTL          | 5 (4.75–5.25) | 0.8      | 2.0      | 0.4      | 2.4      |
| CMOS         | 5 (4.5–6)     | 1.35     | 3.15     | 0.33     | 3.84     |
| LVTTTL       | 3.3 (3–3.6)   | 0.8      | 2.0      | 0.4      | 2.4      |
| LVCMOS       | 3.3 (3–3.6)   | 0.9      | 1.8      | 0.36     | 2.7      |

# Logic Family-Compatibility

**Table 1.5** Compatibility of logic families

|        |         | Receiver |                       |                    |                    |
|--------|---------|----------|-----------------------|--------------------|--------------------|
|        |         | TTL      | CMOS                  | LVTTL              | LVC MOS            |
| Driver | TTL     | OK       | NO: $V_{OH} < V_{IH}$ | MAYBE <sup>a</sup> | MAYBE <sup>a</sup> |
|        | CMOS    | OK       | OK                    | MAYBE <sup>a</sup> | MAYBE <sup>a</sup> |
|        | LVTTL   | OK       | NO: $V_{OH} < V_{IH}$ | OK                 | OK                 |
|        | LVC MOS | OK       | NO: $V_{OH} < V_{IH}$ | OK                 | OK                 |

<sup>a</sup> As long as a 5 V HIGH level does not damage the receiver input

# Power Consumption

- **Power consumption** is the amount of energy used per unit time.
  - Power consumption is of great importance in digital systems.
  - The battery life of systems such as cell phones and laptops is limited by power consumption
- Digital systems draw both **dynamic** and **static** power:
  - ***Dynamic power*** is the power used to charge capacitance as signals change between 0 and 1.
  - ***Static power*** is the power used when the system is idle.

# Power Consumption

- Logic gates and the wires that connect them have **capacitance** (C).
- The **energy** drawn from the power supply to charge capacitance **C** to Voltage  $V_{DD}$  is  $C \cdot V_{DD}^2$ .
- If the voltage on the capacitor switches at **frequency** **f**, it charges the capacitor **f/2** times and it discharges **f/2** times per second.
  - Discharging does not draw energy from the power supply,
- so the **dynamic power** consumption can be given as:

$$P_{\text{dynamic}} = \frac{1}{2} C V_{DD}^2 f$$

# Power Consumption

- Electrical system draw some current even when they are idle.
- When transistors are OFF, they leak a small amount of current,  $I_{DD}$  called the *leakage current* or *quiescent supply current* and is flowing between  $V_{DD}$  and **GND**.
- The **static power** ( $P_{static}$ ) is proportional to this static current and is given as:

$$P_{static} = I_{DD} V_{DD}$$



# Power Consumption

## **Example 1.23** POWER CONSUMPTION

A particular cell phone has a 6 watt-hour (W-hr) battery and operates at 1.2 V. Suppose that, when it is in use, the cell phone operates at 300 MHz and the average amount of capacitance in the chip switching at any given time is 10 nF ( $10^{-8}$  Farads). When in use, it also broadcasts 3 W of power out of its antenna. When the phone is not in use, the dynamic power drops to almost zero because the signal processing is turned off. But the phone also draws 40 mA of quiescent current whether it is in use or not. Determine the battery life of the phone (a) if it is not being used, and (b) if it is being used continuously.

# Power Consumption

**Solution:** The static power is  $P_{\text{static}} = (0.040 \text{ A})(1.2 \text{ V}) = 48 \text{ mW}$ . (a) If the phone is not being used, this is the only power consumption, so the battery life is  $(6 \text{ Whr}) / (0.048 \text{ W}) = 125 \text{ hours}$  (about 5 days). (b) If the phone is being used, the dynamic power is  $P_{\text{dynamic}} = (0.5)(10^{-8} \text{ F})(1.2 \text{ V})^2(3 \times 10^8 \text{ Hz}) = 2.16 \text{ W}$ . Together with the static and broadcast power, the total active power is  $2.16 \text{ W} + 0.048 \text{ W} + 3 \text{ W} = 5.2 \text{ W}$ , so the battery life is  $6 \text{ W-hr} / 5.2 \text{ W} = 1.15 \text{ hours}$ . This example somewhat oversimplifies the actual operation of a cell phone, but it illustrates the key ideas of power consumption.

---

# Exercises

- Write a simulation program for the following conversion:
  - Decimal to Binary, Binary to Decimal, Decimal to Hexadecimal, Binary to Hexadecimal, Decimal to Hexadecimal, Hexadecimal to Decimal